

Exercise2:

```
f=@rho;
%I_builtin=integral3(f,xmin,xmax,ymin,@ymax,zmin,@zmax)
lowerlims = {0,0,0};
upperlims = {8,@ymax,@zmax};
I = gsimp(f,lowerlims,upperlims,10);

zmin=0;zup=@(x,y) x+2*y;
ymin=0;yup=@(x) 2*x;
xmin=0;xmax=8;
new_rho=@(x,y,z) exp(-(x.^2+y.^2+z.^2)./sqrt(1+z.^3));
I_bin=integral3(new_rho,xmin,xmax,ymin,yup,zmin,zup);
perror=100*abs((I-I_bin)/I_bin);
fprintf(['Mass using simpson = %f \n' ...
        'Mass using integral3 = %f \n' ...
        'Percentage error = %f \n'],I,I_bin,perror)
```

```
Mass using simpson = 4.581281
Mass using integral3 = 4.533347
Percentage error = 1.057367
```

The percentage error of 1.06% tell us that result obtained using simpson rule is 1.06% close to the result of integral3.

```
function [v] = rho(X) % density
x=X(1); y=X(2); z=X(3);
v = exp(-(x.^2+y.^2+z.^2)./sqrt(1+z.^3));
end

function [v] = zmax(X)
x=X(1); y=X(2);
v= x + 2*y;
end

function [v] = ymax(X)
x=X(1);
v= 2*x;
end

% multi-dimensional simpson main calling function
% goes from outer to inner (i.e., x to z).
function [I] = gsimp(f,a,b,n)
m=length(a); x=zeros(1,m); k=1;
% convert limits to cell array if numbers are given
if (isnumeric(a))
    a=num2cell(a);
end
if (isnumeric(b))
    b=num2cell(b);
end
% call recursive integration function
I=ksimp(f,x,a,b,n,k);
end
```

```

% recursive simpson function
function [I] = ksimp(f,x,a,b,n,k)
I=0; m=length(a);
if isnumeric(a{k})
    ak=a{k};
else
    ak=feval(a{k},x);
end
if isnumeric(b{k})
    bk=b{k};
else
    bk=feval(b{k},x);
end
h =(bk-ak)/n;
for i=0:n
    x(k)=ak+i*h;
    if (k == m)
        if (i == 0 || i == n)
            I=I+feval(f,x);
        elseif (mod(i,2) == 1)
            I=I+4*feval(f,x);
        elseif (mod(i,2) == 0)
            I=I+2*feval(f,x);
        end
    else
        if (i == 0 || i == n)
            I=I+ksimp(f,x,a,b,n,k+1);
        elseif (mod(i,2) == 1)
            I=I+4*ksimp(f,x,a,b,n,k+1);
        elseif (mod(i,2) == 0)
            I=I+2*ksimp(f,x,a,b,n,k+1);
        end
    end
end
I=I*h/3;
end

```